Lex Technical Manual

0. General remarks

Case and indentation

The Lex language is case and indentation-sensitive. The number of spaces introducing each indentation level is free, but must be consistent throughout the code. As a rule of thumb, we suggest using four spaces for each identation level.

Statements

Lex programs are sequences of statements separated by line breaks. A statement starts with no indentation and can be one of the following:

1. An import statement (see Imports)
2. A section label (see Labels)
3. A type declaration (see Types)
4. A function declaration (see Functions)
5. An event declaration (see Events)
6. A rule declaration (see Rules)
7. A comment (see Comments)

1. Imports

The syntax of imports is

```
import [option] path.to.file

option := `akomaNtoso` | `formex`
```

where `path.to.file` is a sequence of identifiers. These identifiers are converted to a path `import_path := ${root_path}/path/to/file.${extension}` that identifies a file to be imported.

Depending on the option, the behavior of `import` behaves differently: * If no option is specified, then `extension` is set to `.lex` and the content of the file at `import_path` is loaded into the context. * If an option is specified, then `extension` is set to `.xml` and the content of the AkomaNtoso or Formex file is loaded into the context. AkomaNtoso and Formex are standard XML formats for machine-readable legal documents. When importing such files, the text of the law is imported (as comments) to the corresponding parts of the legal formalization by the Lex compiler. The Lex documentation files of the formalization then show both the formalization and the original legal text imported from the XML files.

2. Labels

Labels define the structure of the law, dividing it into chapters, articles, paragraphs, etc.

The syntax of labels is as follows:

```
label_kind [`[` num `]`] quoted_string [quoted_string]

label_kind := `law` | `title` | `chapter` | `section`
            | `article` | `paragraph` | `point` | `subpoint`
```

where `num`denotes an integer ≥ 1 and `quoted_string` denotes a string in double quoted.

Each label defines a part of the corresponding law; depending on the `label_kind`, it can be a full law, a title, a chapter, a section, and article, a paragraph, a point, or a subpoint. This order is fixed (i.e., a title is always a sub-part of a law, a chapter is always a sub-part of a title, etc.). To define intermediate levels in the hierarchy (e.g., subchapters, subparagraphs. . .) , a positive number can be specified in square brackets after the `label_kind`.

Each label can contain two quoted strings. The first quoted string is compulsory and constitutes the label's identifier, often a number or letter. The second quoted string is optional and specifies the label's title.

In Lex, as in the law, there are no explicit markers for ends of laws, articles, etc. The part of the law covered by a label starts with this label and ends when a label of the same or a higher level is reached.

Lex enforces the followings constraints on labels: * Every label other than `law` must be defined within the scope of a `law`. * Every rule must be defined within a `law` and an `article`. * Every identifier at a given level must be unique among the identifiers of the next-higher level. * Identifiers of `article` must be unique within the scope of a `law`.

Examples

> Article 6. Lawfulness of processing

is formalized as

```
article "6" "Lawfulness of processing"
```

> 1. Processing shall be lawful only if. . .
>   (a) . . .
>   (b) . . . Point (f) of the first subparagraph shall not apply. . .

is formalized as

```
paragraph "1"
paragraph[1] "1"
point "a"
...
point "f"
...
paragraph[1] "2"
...
```

with `paragraph[1]` introducing subparagraphs.

3. Types

The syntax of type declarations is

```
`type` ident [`is` (ident | atomic_type)]
[<identation> doc_string]

atomic_type := `bool` | `int` | `float` | `string`
             | `span` | `time` | `money` ident
```

where `ident` is an identifier and `doc_string` is an optional string in triple double quotes (`"""string"""`) giving a high-level description of objects of the given type. Type identifiers are usually lower-case.

The `is` clause specifies a base type for the new type. All constants from the base type can then be embedded in the new type, but elements belonging to the new type are still considered a separated sort.

Lex's atomic types are:

- `bool` for Booleans;
- `int` for integers;
- `float` for floating-point numbers;
- `string` for strings;
- `span` for time spans;
- `time` for timestamps;
- `money c` for money in currency `c`.

Examples

One can declare the type of user IDs, represented as integers in a real-world system, as

```
type user_id is int
    """a user identifier"""
```

One can declare the types of purposes of processing, represented as strings, as

```
type purpose is string
    """a purpose of processing"""
```

One can declare the type of user consent actions, without a concrete base type, as

```
type user_consent_action
```

Recall that both the `is` clause and the documentation string are optional.

4. Functions

The syntax of function declarations is

```
`function` ident
    ident `:` ident
    ident `:` ident
    ...
    ident `:` ident
`->` ident
[<identation> doc_string]
```

where identifiers separated by a colon denote `(arg_type, arg_name)` pairs and the final identifier denotes the output type of the function. Function and argument identifiers are usually lower-case.

Note that Lex function declaration do not have to have an executable body. They merely declare function symbols that can be used within rules.

Examples

Assume two types

```
type individual
type company
```

One can declare a function that returns the age of an individual as “

```
function age
    i : individual
-> int
    """the age of individual {i}"""
```

One can declare the company that an individual works for at a given date (assuming there is only one) as

```
function employer
    i : individual
    d : time
-> company
    """the company employing individual {i} at date {d}"""
```

5. Events

Together with type declarations, event declarations form the backbone of of a legal ontology. There are two kinds of events: *events proper*, introduced with the keyword `event`, that denote real-world actions, and *predicates*, introduced with the keyword `predicate`, that denote relationships between entities.

The syntax of event declarations is

```
[capability] event_kind ident
[<indentation> doc_string]
    ident `:` ident
    ...
    ident `:` ident

event_kind := `event` | `predicate`

capability := `causable` | `causable` `observable`
            | `causable` `suppressable` | `suppressable`
            | `observable` | `internal`
```

where identifiers separated by a colon denote `(arg_type, arg_name)` pairs. Event name identifiers usually use title case while event argument identifiers are lower-case. In documentation strings, one can refer to the value of argument `arg` using the syntax `{arg}`. Documentation strings should describe what concrete fact is being captured by the event declaration.

In an event declaration, the *capability* denotes what we assume to be able to do with the action encoded by the event at runtime in order to ensure compliance. The `observable` capability means that we can only observe the corresponding action, but not prevent it or force it to happen. The `causable` capability means that we can cause, i.e., force the action to be performed for any choice of its parameters. The `suppressable` capability means that we can suppress the action. This requires us to observe the action in the first place, and hence `suppressable` is stronger than `observable`. Causing and observing/suppressing are distinct capabilities, allowing for the combinations `causable observable` and `causable suppressable`. If no capability is specified, then the corresponding action might not even be observable. This is called a *non-observable* event. As a rule of thumb, we can assume most actions to be observable.

A special class of events are *internal events* (and, similarly, *internal predicates*), which are identified by the capability `internal`. An internal event is an event

that is not happening in the real-world, but rather used to refer to a certain fact *defined in terms of concrete actions*. For example, *fulfilling a legal requirement* is a legal fact that is typically defined in terms of certain tangible, more concrete actions. Such a legal fact would typically be defined as an internal event.

Examples

Assuming types

```
type data
    """a piece of data"""

type data_subject
    """an identified or identifiable natural person"""
```

then the definition

> 'personal data' means any information relating to an identified or identifiable natural person ('data subject');

is reflected into the predicate declaration

```
observable predicate PersonalData
    """data {d} is personal data of data subject {ds}"""
    d : data
    ds : data_subject
```

To define data processing of a certain data by a company for a given purpose, one can use the event declaration

```
suppressable event ProcessesData
    """company {c} processes data {d} for purpose {p}
       as part of data processing activity {a}"""
    c : company
    d : data
    p : purpose
    a : activity
```

The capability `suppressable` means that we assume to be able to prevent data processing if this would become necessary to guarantee legal compliance. This can be meaningful in the context of privacy law.

To define a data subject giving consent to a company to use their data for a given purpose, one can use the event declaration

```
observable event GivesConsent
    """data subject {ds} gives consent to company {c}
       to use their data for purpose {p}"""
```

```
ds : data_subject
c : company
p : purpose
```

Note that in this context, it would likely make little sense to make `GiveConsent` causable or suppressable, since causing or suppressing user consent would defy the purpose of data protection.

6. Rules

The core of a Lex formalization is a sequence of rules. We first present the general syntax of rules (a) and discuss the three different types or rules (b). Then, we give a more complete reference of Lex's expression syntax (c) and the optional patterns syntax (d).

(a) Lex rules: General syntax

The general syntax of Lex rules is

```
`rule` [string]
[<indentation> doc_string]
    `whenever`
        expression
        ...
        expression
    verb
        expression | reference
        ...
        expression | reference
[[[`transparently`] `enforceable`]
[(`suppressing` | `causing`) ident, ..., ident]*]

verb := `oblige` | `constitute` | `except`
```

Each rule can be decomposed into five parts:

- The keyword `rule` followed by an optional string giving a name to the rule. Note: a unique for the rule name is **required** as soon as at least two rules are under the same lowest-level label.
- An optional documentation string.
- The keyword `whenever` followed by a sequence of expressions (one by line) that describe the rule's premisses.
- A verb in `oblige`, `constitute` or `except` followed by a sequence of expressions or references (one by line, see the difference between expressions and references below) that describe the rule's conclusions.

- If the verb was `oblige`, an optional clause `transparently enforceable`... or `enforceable`... where the chosen keyword is followed by a list of statements of the form `suppressing Event_1, ..., Event_n` or `causing Event_1, ..., Event_n`. The `enforceable` clause states that the obligation can always be fulfilled by causing or suppressing the corresponding events. The `transparently enforceable` clause additionally states that the obligation is enforceable *and* that causing or suppressing events is only ever necessary when it is certain that the rule will be violated (i.e., enforcement is *precise*).

(b) Rule types

There are three types of rules in Lex: obligation rules, constitutive rules, and exception rules. Each rule type defines a different type of requirement: * Obligation rules define a requirement of the form "if X is the case, then Y **shall** be the case". * Constitutive rules introduce a definition of the form "X **counts as** Y". * Exception rules state "if X is the case, then rules R **shall not be applicable**".

(i) Obligation rules

Obligation rules are introduced by the verb `oblige`. In an obligation rule, each line after `oblige` contains an expression denoting a fact.

If `premiss_1` to `premiss_m` are the premisses of an obligation rule and `conclusion_1` to `conclusion_n` its conclusions, then the rule means "whenever all of `premiss_1` to `premiss_m` are the case, then all of `conclusion_1` to `conclusion_n` must be the case".

Note that all obligation rules defined in a Lex program are by default cumulative, i.e., they must all hold at all times.

Example

Assume event declarations

```
suppressable event ProcessesData
    """company {c} processes data {d} for purpose {p}
       as part of data processing activity {a}"""
    c : company
    d : data
    p : purpose
    a : activity

observable predicate IsLawful
    """data processing activity {a} is lawful"""
    a : activity
```

```
observable predicate IsFair
    """data processing activity {a} is fair"""
    a : activity

observable predicate IsTransparent
    """data processing activity {a} is transparent"""
    a : activity
```

Then the legal requirement (GDPR Art. 5(1))

> Personal data shall be:
>
> 1. processed lawfully, fairly and in a transparent manner in relation to the data subject ('lawfulness, fairness and transparency');

can be formalized as

```
article "5"

paragraph "1"

rule
    whenever
        ProcessesData(c, d, p, a)
    oblige
        IsLawful(a)
        IsFair(a)
        IsTransparent(a)
transparently enforceable suppressing ProcessesData
```

The annotation `transparently enforceable suppressing ProcessesData` notes that by preventing `ProcessesData`only when strictly necessary (= when either lawfulness, fairness, or transparency is violated), we can always ensure that the rule is fulfilled.

(ii) Constitutive rules

Constitutive rules are introduced by the verb `constitute`. In a constitutive rule, each line after `oblige` contains **a single `internal` event or predicate**.

If `premiss_1` to `premiss_m` are the premisses of a constitutive rule and `conclusion_1` to `conclusion_n` its conclusions, then the rule means "whenever all of `premiss_1` to `premiss_m` are the case, then `conclusion_1`to `conclusion_n` hold".

Note that such a constitutive rule generally does not introduce a necessary reason for the conclusions to hold, but only a sufficient condition. There may exist other rules in the code that define other conditions for the same conclusions to hold.

Example

With the event declarations above as well as

```
observable event GivesConsent
    """data subject {ds} gives consent to company {c}
       to use their data for purpose {p}"""
    ds : data_subject
    c : company
    p : purpose

observable predicate PersonalData
    """data {d} is personal data of data subject {ds}"""
    d : data
    ds : data_subject
```

the hypothetical legal requirement

> Processing for a purpose is lawful when the data subject has given consent for its processing for that purpose.

can be formalized as

```
rule
    whenever
        ProcessesData(c, d, p, a)
        PersonalData(d, ds)
        ONCE GivesConsent(ds, c, p)
    constitute
        IsLawful(a)
```

The `ONCE` operator is explained in section (c) below.

(iii) Exception rules

Exception rules are introduced by the verb `except`. In an exception rule, each line after `except` contains a reference to a single rule name or label. If the reference contains a rule name, then an exception is introduced only for the corresponding rule. If the reference contains a label, then an exception is introduced for each rule under this label (e.g., for a full chapter, article, point, etc.). Exceptions can be introduced for obligation rules, but also for constitutive rules and other exception rules.

If `premiss_1` to `premiss_m` are the premisses of an exception rule and `conclusion_1` to `conclusion_n` its conclusions, then the rule means "whenever all of `premiss_1` to `premiss_m` are the case, then rules in `conclusion_1`to `conclusion_n` shall not apply".

Note that such an exception rule generally does not introduce a necessary reason for the conclusions to hold, but only a sufficient condition. There may exist

other rules in the code that define other conditions under which the referenced rules shall not hold.

References to a rule have the form

```
rule "name_of_rule"
```

while references to a label have the form, e.g.,

```
law "name_of_law"
chapter "number_of_chapter"
article "1" paragraph "2"
paragraph "6"
```

The current position in the law is used as root when resolving the labels: e.g., in the above example, `paragraph "6"` would automatically match article 6 *of the current article*.

Example

With the event declarations above as well as

```
observable event LegalWarrant
    """company {c} is required by public authorities
       to process data {d}"""
    c : company
    d : data
```

the hypothetical legal requirement

> Article 5 §1 shall not apply when there exists a legal warrant to use the processed data.

can be formalized as

```
rule
    whenever
        LegalWarrant(c, d)
    except
        article "5" paragraph "1"
```

(c) Expressions

The syntax of expressions is

```
expr := ident `(` term `,` ... `,` term `)`
                                        # Event / predicate
      | term `=` term                   # Equality
      | unop expr                       # Unary operator
      | expr binop expr                 # Binary operator
      | untop [itv] expr                # Unary temporal operator
      | expr bintop [itv] expr          # Binary temporal operator
```

```
      | quant ident. expr              # Quantifier
      | ident `<-` agg `(`ident`;` ident`,` ...`,` ident`;` expr`)`
                                       # Aggregation

term := const                          # Constant
      | ident                          # Variable
      | term_unop term                 # Unary operator
      | term term_binop term           # Binary operator
      | ident `(` term `,` ... `,` term `)`
                                       # Function application

unop := `NOT`
binop := `OR` | `AND` | `IMPLIES` | `IFF`
untop := `PREVIOUS` | `NEXT` | `ONCE` | `EVENTUALLY`
       | `HISTORICALLY` | `ALWAYS`
bintop := `SINCE` | `UNTIL`
quant := `EXISTS` | `FORALL`
agg := `CNT` | `MIN` | `MAX` | `AVG` | `MED` | `SUM` | `STD`

term_unop := `-`
term_binop := `+` | `-` | `*` | `/` | `^` | `=`
            | `<>` | `<` | `>` | `<=` | `>=`

itv := (`[` | `)`) span `,` (span | `*`) (`]` | `)`)

const := `true` | `false` | int | float | string
       | time | span | money
num := (`0`-`9`)+
int := [-] num
float := [-] num+ `.` num*
string := `"` .* `"`
time := `<backquote>` .* `<backquote>`
span := num [`s` | `m` | `h` | `d` | `M` | `y`]
money := `|` <currency_symbol> float `|`
```

Expressions combine events/predicates of the form `Event(term_1, ..., term_n)` and equations of the form `term_1 = term_2` to describe a combination of facts.

To create expressions, Lex provide standard logical operators (`NOT`, `OR`, `AND`, `IMPLIES`, `IFF`) and quantifiers (`EXISTS`, `FORALL`), but also * Temporal operators: These describe sequences of events over time, as in temporal logic. Each temporal operator can be followed by an optional interval of the form `[a,b]`, `(a,b)`, `[a,b)`, `(a,b]`, `(a,*)`, or `[a,*)` (`*` stands for ∞). The available temporal operators are

`PREVIOUS expr`: Denotes that in this instant *immediately preceding the present*, `expr` was the case.

`NEXT expr`: Denotes that in this instant *immediately following the present*, `expr` will be the case.

`ONCE expr`: Denotes that *at some point in the past*, `expr` was the case.

`EVENTUALLY expr`: Denotes that *at some point in the future*, `expr` will be the case.

`HISTORICALLY expr`: Denotes that *at all instants in the past*, `expr` was the case.

`ALWAYS expr`: Denotes that *at all instants in the future*, `expr` will be the case.

`expr_1 SINCE expr_2`: Denotes that *at some point in the past*, `expr_2` was the case, and since that point, `expr_1` has been the case uninterruptedly until the present (included).

`expr_1 UNTIL expr_2`: Denotes that *at some point in the future*, `expr_2` will be the case, and until that point, `expr_1` will be the case uninterruptedly from the present (included).

The interval placed after the operator restricts the time distance between the `expr` (binary: `expr_2`) fact and the present. For instance, `ONCE[0,2d) expr` is true if, and only if, `expr` was true between 0 and 2 days in the past. See below for the description of the time spans used in interval bounds (such as `2d`).

Aggregations: These allow for computing the maximum, the minimum, the average... of a variable whenever some fact is the case, as in SQL. For example, `x <- MAX(a; ; E(a))` sets `x` to be the maximum of the variable `a` such that the event `E(a)` is the case in the present. In an aggregation of the form `x <- agg (y; z_1, ..., z_n; expr)`, `x`is the variable containing the aggregate value, `y` is the variable on which aggregation is performed, `z_1` to `z_n` are optional grouping variables, and `expr` is an expression used to filter relevant values of `y`.

Terms used in event/predicate arguments and equations can be:

- Constants:
  - Integer constants;
  - Float constants (must contain a `.`symbol);
  - String constants (in double quotes);
  - Time constants (in YYYY-MM-dd hh-mm-ss format, between back quotes);
  - Time span constants of the form `num [unit]` where `num` is an unsigned integer constant and unit is an optional time unit in `s` (seconds), `m` (minutes), `h` (hours), `d` (days), `M` (months), `y` (years). Absence of a time unit is interpreted as seconds;
  - Money constants (in `|` , unsigned int preceded by a currency symbol).
- Unary (`-`) or binary (`+`, `-`, `*`, `/`,`^`, `=`,`<>`, `<`, `>`, `<=`, `>=`) operators.

- Function applications.

(d) Patterns

When all premisses or all conclusions of a rule should happen at the same time in the past or the future, a more natural, optional pattern syntax can be used. Patterns can be used after `whenever` and `oblige`, on the same line. Patterns cannot be used after `constitute` and `except`.

The syntax of patterns is

```
pattern := `eventually` [long_future_itv]
         | `once` [long_past_itv]
         | `always` `in` `the` `future` [long_future_itv]
         | `always` `in` `the` `past` [long_past_itv]
         | `eventually` `delaying` `if` expr [long_future_itv]
         | `always` `since` expr [long_past_itv]

long_past_itv := long_itv
               | `before` span
               | `strictly` `before` span

long_future_itv := long_itv
                 | `after` span
                 | `strictly` `after` span

long_itv := `within` span
          | `between` span `and` span
          | `strictly` `between` span `and` span
          | `between` span `and` span `excluded`
          | `between` span `excluded` `and` span
```

Example

> Whenever consent is given by a user, that user's personal data shall be processed at least once within a day

can be formalized as

```
rule
    whenever
        GivesConsent(ds, c, p)
    oblige eventually within 1d
        DataProcessing(c, d, p, a)
        PersonalData(d, ds)
```

7. Comments

There are two kinds of comments in Lex :

- Short comments are introduced by `#`. They can follow non-comment code and span until the end of the current lines.
- Long comments are introduced by the non-indented keyword `note` followed by a string containing the text of the comment. That string can be in double quotes (single-line) or triple double-quotes (multi-line).